Lab 3: Component Modelling & Architectural Pattern Selection

Topic:Public Bus Live Tracking & Crowding Estimator

Name:Nookala Sai Tushar Krishna

SRN:PES2UG24CS331

SEC:F

Scenario: A passenger uses the app to view nearby buses, track their live location and check the estimated crowding level before boarding.

1. Architectural style analysis

Layered Architecture: This is simple and easy to understand. It separates the app, business logic and data. The downside is that scaling individual parts can be harder.

Microservices Architecture: Different services can handle bus tracking, crowd estimation and route data separately. It can scale well and one service failing does not have to stop everything, but it is more complicated to build and maintain.

Client-Server Architecture: The passenger app communicates with a central server. It is simple and keeps bus and route data in one place, but the server can become a bottleneck or single point of failure.

2. Architecture selection

We chose Layered Architecture for the Public Bus Live Tracking & Crowding Estimator.

The main reason is that the system has a clear separation between the passenger app, the tracking and crowding logic, and the stored bus and route data. It also makes the system easier to understand because each part has a specific job.

For security, the data layer can be kept behind the business layer so the passenger app does not directly access the database. For performance, bus locations and crowding values can be processed in the business layer before being shown to users.

3. Main components

Passenger App - shows live bus locations, routes and estimated crowding.

Bus Tracking - receives bus location data and provides current bus positions.

Crowding Estimator - estimates whether a bus has low, medium or high crowding.

Route & Bus Management - handles bus routes, stops, schedules and bus information.

Database - stores bus, route, GPS and crowding information.

4. Main interfaces

Passenger App -> Bus Tracking: Live Location API. Requests current bus positions.

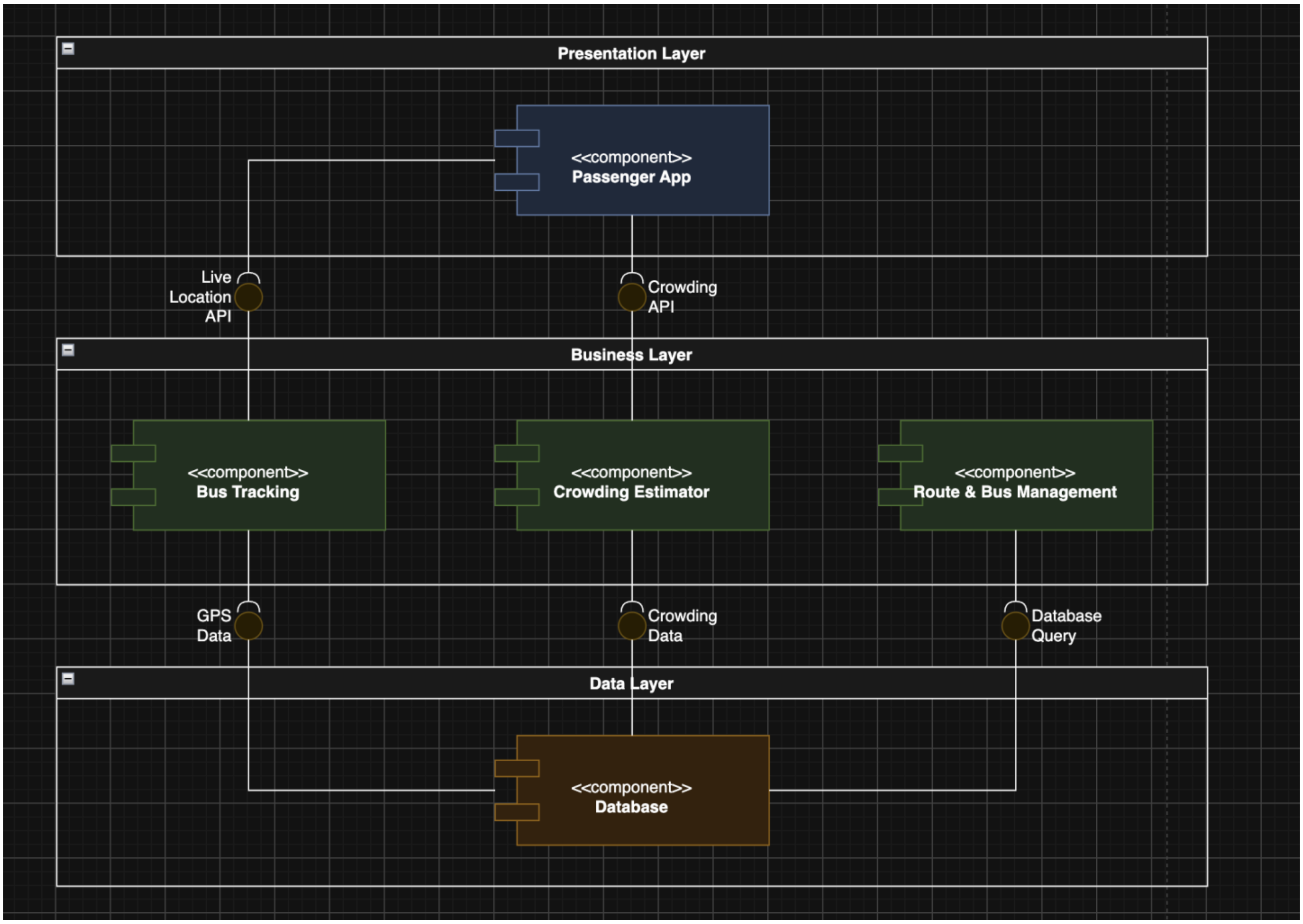
Bus Tracking -> Database: GPS Data. Stores and retrieves bus location information.

Passenger App -> Crowding Estimator: Crowding API. Requests the estimated crowd level of a bus.

Crowding Estimator -> Database: Crowding Data. Stores and retrieves crowding information.

Route & Bus Management -> Database: Database Query. Stores and retrieves route, stop and bus information.

5. Component diagram



The diagram shows the main components and how they communicate. The passenger app gets live bus locations and crowding information. The tracking component stores GPS data, while the crowding estimator works with stored crowding information. Route and bus management uses the database for route and bus details.

This follows the lab requirement to identify components, show interfaces and dependencies, and explain the security and performance effects of the selected architecture.